InsideInsight

Overview

This document covers everything needed to run the InsideInsight Streamlit dashboard locally on any machine.

1. Folder Structure

The dashboard lives in the app/ folder. This is what it looks like:

```
app/
├── app.py                ← main Streamlit app — 4 pages
├── agent.py              ← P4's LangGraph AI agent
├── requirements.txt      ← Python dependencies
├── .env                  ← your API key (never commit this)
├── .env.example          ← template for others
└── sample_data/
    ├── sample_features.csv   ← pricing + competitive scores
    ├── sample_listings.csv   ← listing metadata
    ├── sample_nlp.csv        ← sentiment + topic analysis
    └── sample_reviews.csv    ← guest review text
```

2. Prerequisites

Before running the app you need these installed on your machine:

- Python 3.10 or higher
- pip (comes with Python)
- VS Code (recommended) or any terminal
- A free Groq API key — get one at console.groq.com

To check your Python version:

```
python --version
```

3. Install Dependencies

Navigate to the app/ folder and install all packages:

```
cd app
pip install -r requirements.txt
```

This installs: streamlit, pandas, plotly, langgraph, langchain, langchain-groq, faiss-cpu, sentence-transformers, and all other required packages.

Installation takes 3-5 minutes the first time. You only need to do this once.

If pip install fails
Try: pip install -r requirements.txt --break-system-packages
Or create a virtual environment first:
  python -m venv venv
  venv\Scripts\activate (Windows)
  source venv/bin/activate (Mac/Linux)
  pip install -r requirements.txt

4. Set Up Your Groq API Key

The AI advisor page uses Groq's LLaMA 3.3 70B model. You need a free API key.

Get a free key

1. Go to console.groq.com
2. Sign up for a free account
3. Go to API Keys → Create API Key
4. Copy the key — it starts with gsk_

Add it to your .env file

In the app/ folder, create a file called .env (note the dot at the start):

```
GROQ_API_KEY=your_key_here
```

Important

Never commit the .env file to GitHub.
The .gitignore file excludes it automatically.
Always check git status before pushing to confirm .env is not listed.

5. Run the App Locally

From the insideinsight/ root folder, run:

```
python -m streamlit run app/app.py
```

The app opens automatically at http://localhost:8501 in your browser.

What you should see on startup
Sidebar loads with city dropdown showing: austin, chicago, nashville, seattle, twin_cities
Dashboard page loads with KPI cards for the first listing
Pricing simulator, Health check, and AI advisor pages all navigate correctly
URL paste field in sidebar accepts Airbnb listing URLs

6. Sample Data Files

The app reads from 4 CSV files in app/sample_data/. Here is what each file contains:

File	Rows	Contents
`sample_features.csv`	2,000	listing_id, city, neighborhood, room_type, price, median_neighborhood_price, price_gap_pct, occupancy_rate, occupancy_rank_percentile, amenity_score, review_rank_percentile, competitive_score, peak_month, peak_occupancy_rate
`sample_listings.csv`	2,000	listing_id, city, accommodates, bedrooms, number_of_reviews, review_scores_rating, host_is_superhost, instant_bookable, estimated_occupancy_l365d, estimated_revenue_l365d, amenities_count
`sample_nlp.csv`	2,000	listing_id, city, avg_sentiment_score, pct_positive, total_reviews, cleanliness_mentions, wifi_mentions, checkin_mentions, location_mentions, noise_mentions, host_mentions, value_mentions, sentiment_category, top_praise, top_complaint
`sample_reviews.csv`	500	listing_id, city, review_id, review_date, reviewer_name, comments

To regenerate with all 5 cities

Run this in a Databricks notebook connected to the Gold tables:

```
import pandas as pd

def sample_per_city(df, n=400):
    return df.groupby('city').apply(
        lambda x: x.sample(min(n, len(x)), random_state=42)
    ).reset_index(drop=True)

feat     = spark.table('workspace.default.gold_features').toPandas()
listings = spark.table('workspace.default.gold_listings').toPandas()
nlp      = spark.table('workspace.default.gold_nlp_features').toPandas()
reviews  = spark.table('workspace.default.gold_reviews').toPandas()

feat_sample     = sample_per_city(feat, 400)
feat_ids        = feat_sample['listing_id'].tolist()
listings_sample = listings[listings['listing_id'].isin(feat_ids)]
nlp_sample      = sample_per_city(nlp, 400)
reviews_sample  = sample_per_city(
    reviews[reviews['listing_id'].isin(feat_ids)], 100
)

feat_sample.to_csv('/tmp/sample_features.csv', index=False)
listings_sample.to_csv('/tmp/sample_listings.csv', index=False)
nlp_sample.to_csv('/tmp/sample_nlp.csv', index=False)
reviews_sample.to_csv('/tmp/sample_reviews.csv', index=False)
print(feat_sample['city'].value_counts())
```

Download the 4 files from /tmp/ and replace the ones in app/sample_data/.

7. Dashboard Pages

The app has 4 pages accessible from the sidebar navigation:

Page 1 — Dashboard

The main overview page. Shows for the selected listing:

- 5 KPI cards: nightly price, price gap, occupancy, annual revenue, revenue upside
- Insight bar with revenue upside formula: price gap × occupancy days = annual loss
- Pricing position bar chart vs neighbourhood median and city median
- Score breakdown chart (amenities, reviews, occupancy percentile)
- NLP sentiment score, top praise, top complaint, topic mention chart
- Last 3 guest reviews

Page 2 — Pricing simulator

Interactive price testing tool. Shows for any simulated price:

- Projected annual revenue at that price point
- Difference vs current price
- Revenue curve — price vs annual revenue chart
- Neighbourhood price distribution histogram
- Uses neighbourhood average occupancy as fallback for new listings with no history

Page 3 — Health check

5-dimension traffic light scorecard. Each dimension is green / amber / red:

- Pricing — how far above or below neighbourhood median
- Occupancy — rate vs 30% and 60% thresholds
- Amenities — amenity score vs 40 and 70 thresholds
- Reviews — review rank percentile
- Sentiment — VADER sentiment score
- Improvement priorities — ranked list of highest-impact actions
- Neighbourhood leaderboard — your listing vs top 10 in neighbourhood

Page 4 — AI advisor

Generates 4 grounded recommendations for the selected listing:

- Shows listing data context block — exact numbers the AI uses
- Pricing recommendation — cites price_gap_pct and revenue_upside
- Amenity recommendation — cites amenity_score
- Seasonal recommendation — cites peak_month and peak_occupancy_rate
- Review intelligence — cites top_praise, top_complaint, sentiment score
- Toggle for live LangGraph agent (requires GROQ_API_KEY in .env)

8. AI Agent Integration

The AI advisor page has two modes:

Grounded placeholder mode (default, toggle off)

Uses rule-based logic to generate recommendations from the listing's computed statistics. No API call needed. Works without a Groq key. Every recommendation cites the exact source statistic.

Live LangGraph agent (toggle on)

Calls P4's agent.py which runs a 3-node LangGraph pipeline:

- Node 1 — retrieve: loads listing context, finds 3 comparable listings via FAISS vector search
- Node 2 — analyse: sends context to LLaMA 3.3 70B via Groq API
- Node 3 — recommend: generates 3 grounded recommendations with hallucination validation

The agent is initialised once on first use and cached. First run takes 20-30 seconds to build the FAISS index. Subsequent runs are faster.

To enable the live agent

1. Make sure GROQ_API_KEY is set in app/.env
2. Make sure agent.py is in the app/ folder
3. Go to AI advisor page
4. Toggle on 'Use live AI agent'
5. Click Generate AI action plan

9. Sidebar Filters

The sidebar has two ways to find a listing:

URL paste (recommended for demo)

Paste any Airbnb listing URL into the text field. The app extracts the listing ID automatically and pre-selects it. Example:

```
https://www.airbnb.com/rooms/26259302
```

Manual filters

- City — select from austin, chicago, nashville, seattle, twin_cities
- Neighbourhood — filters to neighbourhoods in that city
- Room type — entire home, private room, shared room, hotel room
- Price range slider — filters by nightly price
- Min bedrooms — Any, 1, 2, 3, 4
- Superhost only — filters to superhost listings
- High occupancy only — filters to listings with >60% occupancy rate
- Instant bookable only — filters to instant bookable listings

10. Competitive Score Formula

The 0-100 competitive score is computed by P2 (Bhavisha) in the gold_features table. Here is how it works:

```
competitive_score =
    0.30 × price_gap_norm        # lower gap = higher score (underpriced =
advantage)
  + 0.25 × amenity_score_norm    # relative to neighbourhood max
  + 0.25 × review_rank_pctile    # percentile rank by review count
  + 0.20 × occupancy_rank_pctile # percentile rank by occupancy rate
```

Score tiers:

- 80-100: Market leader — top tier in neighbourhood
- 40-79: Mid-tier — specific improvements available
- 0-39: Fixer-upper — needs attention

Revenue upside formula:

```
revenue_upside = abs(price - median_neighborhood_price) × occupancy_days

Example: abs($98 - $120) × 255 days = $5,610/year recoverable
```

11. Best Demo Listing IDs

Pre-load these in the URL field before the demo. Type the listing ID into the sidebar URL field and press Enter.

City	Listing ID	Price gap	Upside/yr	Best for
Austin	26259302	-76%	$77,380	Biggest upside
Chicago	36141541	-65%	$51,830	Low amenity story
Nashville	994934741947505807	-72%	$45,260	Seasonal story
Seattle	18358544	-60%	$50,735	Pricing simulator
Twin Cities	632566632777514546	-78%	$50,370	Local audience

12. Troubleshooting

App won't start

- Check Python version: python --version (needs 3.10+)
- Run: pip install -r requirements.txt again

- Make sure you're running from the insideinsight/ root folder, not inside app/

No listings showing

- Check that all 4 CSV files are in app/sample_data/
- Try resetting the price range slider to 0-1000
- Select 'All' for neighbourhood and room type

AI agent fails

- Check that GROQ_API_KEY is in app/.env
- Check that agent.py is in the app/ folder
- Toggle off live agent — grounded recommendations still work without Groq

Occupancy shows 0 days

- This listing has no booking history in the dataset
- The app automatically uses neighbourhood average occupancy as fallback
- This is expected for newer or less active listings

Only one city showing

- Your sample CSVs may only have Chicago data
- Regenerate the CSVs using the Databricks notebook in Section 6
- Replace the files in app/sample_data/ with the new one